

Guía explicativa de Arquitectura de Software

1. ¿Qué es arquitectura de software?

La arquitectura de software no es solo dibujar diagramas bonitos. Es la disciplina que decide:

- cómo organizar el código,
- qué responsabilidades tiene cada parte,
- cómo se comunican los módulos,
- cómo se protege la lógica de negocio,
- cómo el sistema evoluciona sin romperse.

Una buena arquitectura hace que el software sea comprensible, seguro, mantenible y adaptable.

2. Regla central del curso

No se trata de “usar la mejor tecnología”. Se trata de resolver el problema correcto con la solución proporcional.

Un sistema pequeño no necesita microservicios. Un sistema grande sí puede necesitar límites claros, eventos y separación de responsabilidades.

3. La diferencia entre problema esencial y detalle incidental

- Problema esencial: lo que realmente importa para el negocio y el usuario.
- Detalle incidental: tecnología, librerías, framework, forma exacta de implementar una funcionalidad.

Ejemplo:

- Esencial: "el usuario debe poder reservar un servicio en 5 segundos con seguridad".
- Incidental: "usar Java, Node, Spring, Nest, PostgreSQL, Redis o MongoDB".

4. Conceptos clave

4.1 Acoplamiento

Es el nivel de dependencia entre módulos. Cuanto más acoplado está el sistema, más difícil es cambiarlo.

4.2 Cohesión

Es el grado en que una pieza del sistema tiene una sola responsabilidad clara.

4.3 Dependencias

Las dependencias deben ir en la dirección correcta: las reglas del negocio no deberían depender de la base de datos ni de una API externa.

4.4 Cambio y evolución

La arquitectura debe facilitar los cambios futuros sin destruir el sistema actual.

5. Cómo se evalúa en este curso

Cada actividad busca demostrar que el estudiante:

1. entiende el problema,

2. define criterios de calidad,
3. toma decisiones justificadas,
4. protege el dominio,
5. valida con pruebas y evidencia.

6. Formato de entrega recomendado

Cada actividad debe incluir:

- README con resumen ejecutivo,
- carpeta docs con diagramas y análisis,
- evidencia de evolución con commits,
- video de sustentación técnica,
- estructura clara en GitHub.

7. Proyecto obligatorio del curso

El problema del proyecto es único y obligatorio para todo el curso:

- una plataforma compleja de gestión logística de última milla para una empresa de retail digital,
- con pedidos, inventario, rutas, repartidores, incidencias, soporte al cliente y sistemas externos.

No se acepta que cada estudiante elija una app distinta. La intención del curso es trabajar sobre la misma arquitectura de negocio con suficiente complejidad para analizar decisiones reales de diseño, consistencia, integración y evolución.

La clave no es la simpleza del problema, sino la claridad del análisis arquitectónico sobre un sistema realista y complejo.

8. Preguntas que debes responder siempre

- ¿Qué problema resuelve este sistema?
- ¿Qué usuarios lo usan?
- ¿Qué restricciones tienen?
- ¿Qué calidad exige el negocio?
- ¿Qué cambia con el tiempo?
- ¿Qué partes son más vulnerables?
- ¿Qué decisión es reversible y cuál no?

9. Mensaje final

Arquitectura no es solo "teoría". Es convertir decisiones de negocio en un sistema que pueda soportar cambios reales. Si tu trabajo demuestra comprensión del problema, justificación técnica y evolución controlada, ya estás pensando como arquitecto.